MIJ TECHNOLOGIES • PYTHON PROGRAMMING SERIES • SERIES 2

ASSIGNMENT 5

Student Records Manager — MIJ Academy Client Project

TOTAL MARKS	DIFFICULTY	ESTIMATED TIME	MODULES TESTED
74 Marks	File Handling (5/5)	5 - 7 hours	Modules 1 - 7

RESTRICTION: *Solve this assignment using topics from Modules 1–7 only (Getting Started through File Handling). This is the **Series 2** edition of Module 7's Assignment 5 — a separate, client-brief-style version of the file handling wrap-up project, distinct from the original Assignment 5. It is built directly from a real client brief rather than a generated specification, so the required file names, functions, and acceptance test below must be followed exactly as stated.*

OVERVIEW

Client: MIJ Academy, a small training centre in Kampala. **Contact:** the Academy Registrar.

"We track our students on paper. Every term we lose records, miscalculate grades, and spend hours working out who our top student is. We need a simple program that runs on the office computer, remembers every student between sessions, works out grades automatically, and gives us a summary report. The staff are not technical, so it must be menu-driven and must never crash on them."

Deliver one Python program, **MODULE 7 WRAP UP PROJECT.py**, that uses a CSV file as the main record store, a JSON file for an exported summary report, **pathlib** to check whether files exist, **csv/json** to read and write them, exception handling around every file operation, and one function per feature plus a **main()** that runs the menu.

Score	Grade
90 and above	A
80 to 89	B
70 to 79	C
60 to 69	D
below 60	F

The grade is never typed in by the user. It is always computed from the score.

PART A — PROGRAM ARCHITECTURE, FILE INITIALIZATION & EXCEPTION HANDLING — 10 MARKS

- Record store: **records.csv**, columns in order: name, age, score, grade
- Summary export: **summary.json**, formatted for humans to read
- Use **pathlib** to check whether **records.csv** exists
- On startup: if **records.csv** does not exist, create it with the header row; if it exists, leave it untouched
- Wrap every file operation in exception handling covering at least **FileNotFoundError** and **OSError** — the program must never crash on the staff
- One function per feature, plus a **main()** that runs the menu

PART B — GRADING RULE & MENU-DRIVEN INTERFACE — 6 MARKS

Implement a loop that shows these options and keeps running until the user quits:

- 1. Add Student • 2. View All Students • 3. Search by Name
- 4. Update Score • 5. Delete Student
- 6. Export JSON Summary • 7. View JSON Summary • 8. Quit
- An invalid choice shows a message and returns to the menu
- Quitting shows a goodbye message

PART C — ADD STUDENT — 10 MARKS

- Ask for name, age, and score
- Compute the grade from the score using the grading rule
- Save the record to **records.csv** immediately, so nothing is lost if the power goes off
- Confirm with the student's name and grade

PART D — VIEW ALL STUDENTS — 8 MARKS

- Read from the file and show a formatted table: rank/number, name, age, score, grade
- Show a line with the total number of students
- If there are no records, say so instead of showing an empty table

PART E — SEARCH BY NAME — 8 MARKS

- Ask for a search term
- Matching must be case-insensitive and must find partial matches (e.g. “ali” finds “Alice”)
- Show every match, or a clear “no match” message

PART F — UPDATE SCORE — 10 MARKS

- Ask for a student's name (case-insensitive) and a new score
- Update the score, recompute the grade, and rewrite the file
- If the student doesn't exist, say so and change nothing

PART G — DELETE STUDENT — 8 MARKS

- Ask for a name (case-insensitive)
- Remove that student and rewrite the file
- If nobody matches, say so and change nothing

PART H — EXPORT & VIEW JSON SUMMARY — 14 MARKS

Export JSON Summary: compute statistics from the records and save them to **summary.json**, formatted for humans to read. The summary must contain:

- **total**: number of students
- **average**: average score, rounded to 2 decimal places
- **top_student**: name and score of the highest scorer
- **low_student**: name and score of the lowest scorer
- If there is no data, say there is nothing to export instead of crashing

View JSON Summary: load **summary.json** and display it. If the file hasn't been created yet, tell the user to export first.

```
average = round(sum(scores) / total, 2)
top_student = student with max(score)
low_student = student with min(score)
```

PERSISTENCE

Close the program, run it again, and every student is still there.

ACCEPTANCE TEST (VERIFIED)

The Registrar will run this exact sequence. Your program must produce these results. If every step passes, the client signs off.

ACCEPTANCE TEST SEQUENCE

```
1. Delete records.csv, start the program -> file recreated, header only
2. Add: Alice Nakato 20/92, Okello Bob 22/78, Atim Carol 19/85,
Kato David 21/58
Expected grades: A, C, B, F
3. View All shows 4 students with those grades
4. Search 'ali' finds Alice. Search 'ALICE' also finds her.
Search 'zzz' shows no match.
5. Export summary. Expected:
total=4, average=(92+78+85+58)/4 = 78.25
top=Alice Nakato 92, low=Kato David 58
6. Update Kato David's score to 64 -> grade becomes D
Export again: average=(92+78+85+64)/4 = 79.75
7. Delete Okello Bob. View All shows 3 students.
Export again: average=(92+85+64)/3 = 80.33
8. Quit and restart -> View All still shows the same 3 students
9. Choose menu option 9 (invalid) -> message appears, menu returns
Delete a name that doesn't exist -> message appears, nothing changes
```

SAMPLE TERMINAL OUTPUT

TERMINAL -- PART 1 OF 2

```
================================================================================
   MIJ ACADEMY -- STUDENT RECORDS MANAGER
================================================================================
records.csv not found -- creating with header row.

1. Add Student   2. View All Students   3. Search by Name
4. Update Score  5. Delete Student      6. Export JSON Summary
7. View JSON Summary                    8. Quit
Choose an option: 1
Name: Alice Nakato | Age: 20 | Score: 92
Saved. Alice Nakato -- Grade: A

Choose an option: 1
Name: Okello Bob | Age: 22 | Score: 78
Saved. Okello Bob -- Grade: C

Choose an option: 2
================================================================================
   ALL STUDENTS
================================================================================
#   Name            Age   Score   Grade
1   Alice Nakato    20    92      A
2   Okello Bob      22    78      C
3   Atim Carol      19    85      B
4   Kato David      21    58      F
Total students: 4

Choose an option: 3
Search term: ali
Match found: Alice Nakato (Age 20, Score 92, Grade A)
```

TERMINAL -- PART 2 OF 2

```
================================================================================
   EXPORT JSON SUMMARY
================================================================================
{
  "total": 4,
  "average": 78.25,
  "top_student": {"name": "Alice Nakato", "score": 92},
  "low_student": {"name": "Kato David", "score": 58}
}
Summary exported to summary.json

Choose an option: 4
Name: kato david | New Score: 64
Updated. Kato David -- new Grade: D

Choose an option: 6
Summary exported. average = 79.75

Choose an option: 5
Name: okello bob
Deleted. Okello Bob removed from records.csv

Choose an option: 2
#   Name            Age   Score   Grade
1   Alice Nakato    20    92      A
2   Atim Carol      19    85      B
3   Kato David      21    64      D
Total students: 3

Choose an option: 9
Invalid choice. Please select 1-8.

Choose an option: 5
Name: zzz
No student found matching 'zzz'. Nothing changed.

Choose an option: 8
Goodbye! Your records are saved in records.csv.
================================================================================
```

BONUS EXTENSIONS (OPTIONAL, AFTER SIGN-OFF)

- Reject scores that aren't numbers instead of crashing (catch ValueError)
- Back up records.csv to records_backup.csv before every update or delete
- Sort the View All table by name or by score

MARKING SCHEME

Part	Marks	Description
A	10	Architecture: file initialization via pathlib, exception handling (FileNotFoundError, OSError), one function per feature
B	6	Grading-rule function and menu loop with invalid-choice handling
C	10	Add Student: input, computed grade, immediate save, confirmation
D	8	View All Students: formatted table, total count, empty-state message

Part	Marks	Description
E	8	Search by Name: case-insensitive partial matching
F	10	Update Score: lookup, recompute grade, rewrite file, not-found case
G	8	Delete Student: lookup, rewrite file, not-found case
H	14	Export & View JSON Summary: stats, human-readable JSON, edge cases
TOTAL	**74**	**100% requires every part working end-to-end with correct calculations**

100% is awarded only if the program passes every step of the Acceptance Test above exactly as specified, including the persistence check (restart and re-verify) and the two negative-path checks (invalid menu option, deleting a nonexistent student).